Lottie Squeeze: Compression Engine Audit

Code review and measured benchmark of the browser-based Lottie compressor · 18 October 2026

> **Verdict: Not ready for professional release yet.**
>
> - **What works:** the default settings give real, measurable savings (20–89% on the test files). On the vector-only test files, the output is visually indistinguishable from the original.
> - **What's wrong:** four bugs can silently damage files: a broken animation, lost transparency, garbage output at low precision, and broken mobile theming.
> - **Effort to fix:** all four are small, low-risk fixes in one file. After those fixes the tool moves to **Ready with limitations**.

No application code was changed during this audit. Every finding comes from reading the code and running the real engine.

How the testing was done

- **Harness:** `audit/harness.html` loads the actual app in a hidden frame and calls its own `optimizeLottie()`, `readDotLottie()` and `handleLottieFile()`. It does not use a copy of the logic.
- **Visual check:** each original and compressed animation is rendered with lottie-web 5.12.2 (canvas renderer), and every frame is compared pixel by pixel. Rendering the original against itself gives exactly 0 difference, so every difference reported comes from the compressor.
- **Score:** "% pixels changed" is the share of pixels in the worst frame that differ by more than 16 out of 255 in any colour channel.

> **Coverage gap.** No real After Effects or LottieFiles exports were available. The five test files are synthetic, generated to look like AE exports (numbers with 15–17 digits). Not covered: text layers, effects, 3D layers, audio, the SVG renderer (used by the preview), and Android/iOS players. Treat every number in this report as measured on these test files only.

A. What actually works

Feature	Status	Evidence
Whitespace minification	Works	Output is always single-line JSON
Decimal precision setting	Works, but rounds every number the same way (B3)	`cleanTree()`, line 576
Remove metadata and names (`meta, nm, mn, ix, cix, cl, ln`)	Works; skipped automatically when expressions are found	179 → 0 names in the multi-layer test; names kept in the expression test
Remove hidden layers and shapes	Works and keeps parent and matte layers, but breaks files with expressions (B1)	29 removed in the multi-layer test, 0.02% pixels changed
Remove unused precomps and images	Works, including nested precomps	1 unused precomp removed
Masks and track mattes kept	Verified	Gradients and masks test: pixel-identical at precision 3

Re-compress embedded images (WebP/JPEG, downscaling)	Works, only keeps the new image if smaller; Auto mode loses transparency (B2)	Images test: 280 KB → 30 KB
Read `.lottie` files (both folder layouts)	Works, images get embedded	10 ms to read a 210 KB file
Bezier path optimization	Not implemented	No code for it
Keyframe or easing optimization	Not implemented	No code for it
Removing default values	Not implemented	No code for it

In short, the engine is a solid minifier with number rounding, structural cleanup and image re-encoding. It does no real animation-data optimization yet.

B. Critical issues

B1. Hidden layers are deleted even when expressions use them CRITICAL

- **Cause:** `removeHidden()` (line 518) never checks whether the file has expressions.
- **Test:** an expression reading `thisComp.layer('Controller')` lost its hidden controller layer. Result: 13.9% of pixels wrong, and the card no longer rotated.
- **Why it matters:** hidden controller layers driven by expressions are common in professional rigs.

B2. "Auto" image mode destroys transparency CRITICAL

- **Cause:** `hasAlpha()` (line 503) only checks one pixel in four, so it misses transparency.
- **Test:** a photo with a scattered transparent edge went from 9,600 transparent pixels to 0. The holes became solid dark green, saved as JPEG.
- **Same check elsewhere:** it also decides whether a JPEG gets flattened onto white, and whether the WebP fallback keeps transparency.

B3. Low precision settings produce garbage, and the slider allows them CRITICAL

Cause: the slider goes down to 0 (line 253), and `cleanTree()` rounds every number the same way.

	Original	Precision 1	Precision 0
Colour	`[0.133, 0.451, 0.941]`	`[0.1, 0.5, 0.9]`	`[0, 0, 1]` (pure blue)
Overshoot easing (out x)	0.473	0.5	0 (easing gone)
Frame rate	29.97	30	30
End frame	89.91	89.9	90 (one extra frame)
Time remap end	2.966 s	3 s	3 s
Worst-frame % pixels changed	—	0.5–8.5%	17–31%

Frame rate, frame times and time-remap values should never be rounded like positions are.

B4. Name stripping is on by default and silently breaks developer features CRITICAL

- It removes every layer name.
- **What breaks:** colour and property changes done from code on Android, iOS and React Native (they find layers by name); any code that looks up layers by name; CSS that targets layer IDs or classes in lottie-web.
- **Why it's easy to miss:** none of this shows up in the preview, so the option looks harmless. Expression detection only protects files that use expressions.

B5. Size figures that mislead MEDIUM

- **Example animation:** the "96.7% saved" compares against pretty-printed JSON (line 688), which was 4× bigger than minified JSON in the test files.
- **`.lottie` input:** "before" is the zipped file size (line 753) while "after" is unzipped JSON. With images kept, a 210 KB `.lottie` becomes 280 KB of JSON, so the page shows about +33% instead of a saving.
- **No `.lottie` output:** the result cannot be saved back as `.lottie`.
- **No gzipped baseline:** only the gzipped size of the output is shown, not of the original.

B6. Re-compressing an already compressed file keeps degrading its images MEDIUM

- Three passes in a row raised the changed pixels from 6.1% → 7.8% → 9.0%.
- **How it happens:** the "Optimize further" button on the video tab, or someone re-uploading their output.

B7. Wrong property count in shape groups after removing hidden shapes LOW

- 28 of 29 groups ended up with a property count (`np`) that no longer matches their contents.
- lottie-web ignores it (no visual difference); other players are untested.

C. Missing capabilities

1. **Smarter rounding per property type:** e.g. 2 decimals for positions and paths, 3 for colours and easing, never rounding times or frame rate.
2. **Keyframe optimization:** turn an animated property whose keyframes are all identical into a static value; drop redundant middle keyframes; drop easing on the last keyframe; drop zero spatial tangents.
3. **Removing default values** such as `hd:false` , `bm:0` , `ddd:0` , `ao:0` (defaults need checking per player first).
4. **Merging duplicate images and precomps.**
5. **Saving as `.lottie` :** a zip with images as separate files, usually the smallest format for delivery.
6. **Removing dead layers:** layers entirely outside the timeline, or at 0% opacity the whole time.
7. **Background processing (Web Worker),** cancelling stale runs, and caching decoded images.
8. **A built-in "Verify" step** that compares frames like the harness does and shows a quality score.
9. **Gzipped sizes before and after,** since servers send files gzipped.

Compared with LottieFiles: their optimizer was not run, because that would mean uploading test files to their service, so no parity or comparison is claimed. Publicly, their tool is described as offering precision reduction, removing unused data and converting to `.lottie` . Of those, this tool lacks `.lottie` output and per-property rounding.

D. Benchmark results (measured)

Default settings (precision 3, all options on, WebP quality 75)

Test	Original (minified)	Output	Saved	Gzipped before → after	Time	Plays	Worst-frame % px changed
Simple vector	2,398 B	1,915 B	20.1%	665 → 469 B (29.5%)	0.3 ms	Yes	0.018%
31 layers + precomp	59,207 B	34,486 B	41.8%	11.7 → 4.9 KB (58.4%)	1.8 ms	Yes	0.022%
Gradients + masks + matte	5,861 B	3,753 B	36.0%	1.75 → 1.00 KB (42.8%)	0.2 ms	Yes	0 (identical)
Embedded images	280,360 B	29,962 B	89.3%	211 → 21 KB (90%)	28 ms	Yes	6.06% (lossy WebP)
Character (36 animated paths, 29.97 fps, expression)	422,617 B	192,695 B	54.4%	174 → 56 KB (67.7%)	8.9 ms	Yes	0.039%

Precision vs quality (worst-frame % pixels changed)

Test	Precision 3	Precision 2	Precision 1	Precision 0
Simple	0.018	0.24	0.51	17.2
Multi-layer	0.022	2.16	8.47	21.7
Gradients + masks	0	0.33	0.80	31.4
Character	0.039	0.20	0.99	20.0

- **Size gain from lower precision:** going from precision 3 to 2 saved only 1–4 more percentage points of size.
- **Rounding alone:** with precision 5 and everything else off, savings were 0.8–47%, almost entirely from trimming long decimals; renders were practically identical.

Lossless vs lossy

- **No visual change:** minifying and removing metadata, names and unused assets. Removing hidden layers also, except in files with expressions (B1). Removing names does break developer features (B4).
- **Lossy:** precision rounding and image re-encoding.

Large file and error handling

Measurement	Result
Test file	16.4 MB, 1,479 layers
Read (JSON parse)	44 ms
Optimize	234 ms; output 7.4 MB

Page freeze in the real upload flow	0.8 s (longest gap 797 ms)
Memory growth	about 87 MB
Threading	All on the main thread; every slider change re-runs the whole pipeline, including image re-encoding
Invalid JSON / non-Lottie file	Clear messages
Corrupt `.lottie`	Shows a raw technical zip error

E. Recommended changes (prioritized)

All changes are in `lottie-squeeze.html`; the standalone copy is generated from it.

#	Change	Functions	Impact	Effort	Risk
P1-1	Skip hidden-layer removal when expressions are found; keep layers used as mattes (`tp`); fix property counts	`removeHidden()`, `optimizeLottie()`	Fixes B1, B7	Low	Low
P1-2	Check every pixel for transparency, stopping at the first one found	`hasAlpha()`	Fixes B2	Low	Low
P1-3	Round each property type differently; never round frame rate or times; min 3 decimals for colours and easing; slider minimum 1	`cleanTree()`, `#opt-prec`	Fixes B3	Low–Med	Low
P1-4	Split names option: "Remove editor metadata" (safe, on) and "Remove layer names (breaks keypaths)" (off)	markup, `optOpts()`, `cleanTree()`	Fixes B4	Low	Low
P1-5	Honest sizes: compare against minified JSON, handle `.lottie` correctly, show gzipped before and after	`loadOptSource()`, `handleLottieFile()`, `runOptimize()`, `renderLog()`	Fixes B5	Low	None
P1-6	Don't re-encode images already WebP/JPEG unless the format or scale changes	`recompressImages()`	Fixes B6	Low	Low
P2-7	Keyframe optimization pass, checked with the harness	new `optimizeKeyframes()`	Bigger savings	Med	Med
P2-8	Merge duplicate images and precomps	`removeUnusedAssets()`	Bigger savings	Low	Low
P2-9	Save as `.lottie` (zip library already loaded)	new `saveDotLottie()`	Biggest delivery win	Med	Low
P2-10	Lossless / Balanced / Maximum presets	UI + `optOpts()`	Usability	Low	Low
P3-11	Background processing, cancelling stale runs, image cache	`runOptimize()`, new worker	No freezes	Med	Med
P3-12	Built-in "Verify" quality score	harness logic	Trust	Med	Low
P3-13	Friendly error for corrupt `.lottie`	`handleLottieFile()`	Polish	Low	None

P3-14	Test with real AE / LottieFiles exports, and on Android and iOS	—	Confidence	—	—

Compression presets (after the fixes)

Mode	What it does	Trade-off
Lossless	Only changes that don't alter how the animation looks; names and timing kept	Smallest savings. The 0.8–47% above came from precision-5 rounding, so strict no-rounding savings will be lower (not measured).
Balanced (default)	Per-property rounding, keyframe optimization, WebP quality 80 that keeps transparency, names kept	Should match or beat today's precision-3 results with less risk (not measured yet).
Maximum	Paths at 1 decimal, images at quality 60 and 75% size, names removed, `.lottie` output	Smallest files. Can show on heavily scaled-up precomps; images get softer.

F. Production readiness

Stage	Rating	Why
Now	Not ready for professional use	B1 and B2 silently damage output with settings people will actually use; B4 breaks mobile developer features by default.
After P1 (six small fixes)	Ready with limitations	Safe for lottie-web on the web; verified on synthetic files only.
Professional grade	Needs P2-7, P2-9, P3-14	Keyframe optimization, `.lottie` output, and testing with real AE exports and mobile players.

The harness is reproducible (fixed random seed), so the same benchmark can be re-run after each fix to prove nothing got worse.

Lottie Squeeze · Compression engine audit · Measurements from `audit/harness.html`, lottie-web 5.12.2 canvas renderer, Chromium, synthetic test files.